DATA ENGINEERING

Elasticsearch Visual Guide

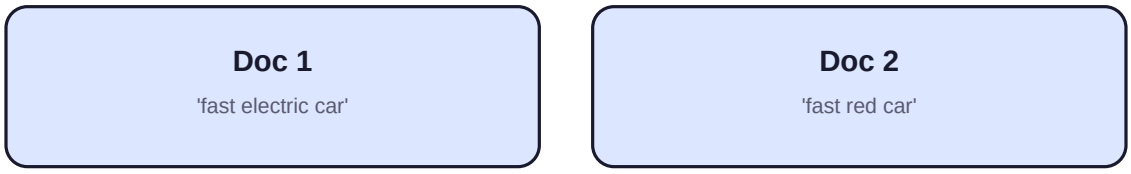
Search at scale.
Inverted indexes, shards, and relevance.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Inverted Index

Like a book index: term → list of documents containing it.



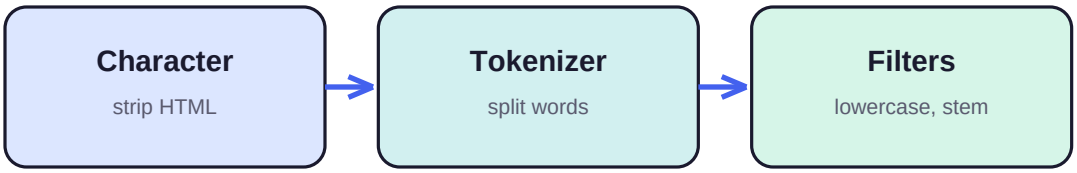

```
fast     → [doc1, doc2]
electric → [doc1]
car      → [doc1, doc2]
red      → [doc2]
```

Search = look up each query term's posting list, intersect, rank. No table scans — that's why full-text search is instant at billions of docs.

STEP 2/6

Analysis: Text In, Terms Out

The analyzer pipeline decides what gets indexed.



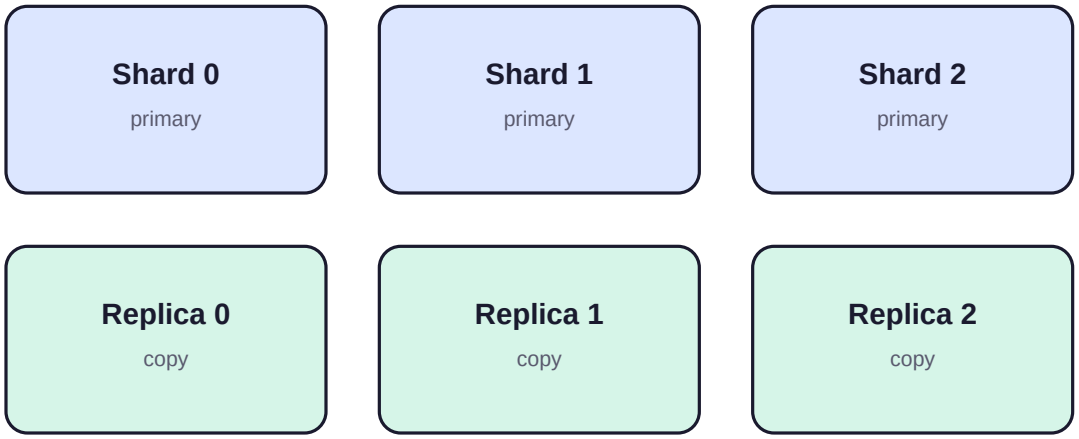

```
"Running Shoes!" → running | shoes
                 → run | shoe   (stemmed, lowercased)
```

Query and index MUST use the same analyzer or terms won't match. Custom analyzers per language/field are the norm in production.

STEP 3/6

Shards & Replicas

An index is split into shards; each shard has replicas.



Shards = horizontal scale (more shards, more parallel search). Replicas = availability + read throughput. Rule: shards sized 10-50GB each.

- Too many shards: overhead per shard kills the cluster (shard sprawl)
- Replicas never sit on the same node as their primary
- Rollover: time-based indices (logs-2026.10.09) keep shards right-sized

STEP 4/6

Relevance Scoring (BM25)

Not just matching — ranking by what matters most.

BM25 scores each doc on three signals:

- Term frequency: more mentions = more relevant (with diminishing returns)
- Inverse document frequency: rare terms ('elasticsearch') weigh more than common ones ('the')
- Field length: a match in a short title beats a match in a long body

Tune with field boosts (title^3) and function scores (recency, popularity). Relevance is a product decision — A/B test it like one.

STEP 5/6

Mappings: Schema for Search

Define field types deliberately — dynamic mapping guesses wrong.

```
{"mappings": {"properties": {
  "title":   {"type": "text"},
  "status":   {"type": "keyword"},
  "price":    {"type": "scaled_float"},
  "location": {"type": "geo_point"}
}}}
```

text = analyzed, full-text search. keyword = exact match, aggregations, sorting. Pick wrong and your filters silently return nothing.

- Multi-fields: index a name as both text and keyword
- Changing a mapping usually means reindexing — plan for it
- Index templates auto-apply mappings to new time-based indices

STEP 6/6

Aggregations: Analytics on Search

Facets, histograms, and metrics — computed across shards.

Bucket aggs

terms, date_histogram, ranges

Metric aggs

avg, sum, percentiles

One request returns search hits AND 'avg price by brand' AND 'histogram over time'. This is how faceted navigation (Amazon's sidebar filters) works at scale.

Aggregations run distributed: each shard computes partials, the coordinator merges. Deep pagination + huge cardinality = memory pressure — use composite aggs.

STEP ✓/✓

Key Takeaways

Elasticsearch — Remember

- The inverted index maps terms to documents — search is posting-list intersection, not table scans.
- Analyzers (character → tokenizer → filters) must match between index and query time.
- Shards scale search horizontally (10-50GB each); replicas add availability and read throughput.
- BM25 ranks by term frequency, rarity (IDF), and field length — boost fields deliberately.
- Map text vs keyword correctly; aggregations turn search into real-time analytics.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev